

Arrays, 2D/3D, and ArrayLists

My teaching notes

A word before you read. These are my notes from over the years, cleaned up so they are readable. A few things to keep in mind:

- **They are not a study guide, and they are not gospel.** Not everything here appears in a lecture, and not every topic has notes yet; I will write more as we go.
- **I write from memory and revise every semester.** A typo or an imprecise statement can survive longer than it should.
- **I use AI to help** convert documents to markdown, check for issues, and add explanations. I do not trust it completely, so I am still editing these myself.
- **Java changes.** The basics we cover are pretty resilient, but some features do move over time.

So: if you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. When in doubt, the textbook is the tiebreaker, and the compiler beats us both.

1. One-dimensional arrays: the part you already know (mostly)

An array is a fixed-length, contiguous block of same-typed slots. The two things people gloss over are worth slowing down on.

Declaring a variable is not the same as allocating the array.

```
int[] scores;           // a variable that can refer to an int[]; refers to nothing
    yet
scores = new int[5];     // NOW there is storage: five ints, all defaulted to 0
```

Until that second line runs, `scores` is `null`. Touching `scores[0]` before then is a `NullPointerException`, not an out-of-bounds error. Different bug, different message.

Default values matter. A freshly `new`'d array is not full of garbage; Java zeroes it: `0` for numeric types, `false` for `boolean`, `'\u0000'` for `char`, and `null` for any object type. That last one is the trap: `new String[3]` is three `null`s, not three empty strings.

Initializer shortcuts:

```
int[] a = {10, 20, 30};           // only valid at declaration
int[] b = new int[]{10, 20, 30};  // needed if you're not declaring on the same
    line
```

`length` is a field, not a method: `a.length`, no parentheses. (Contrast with `String`'s `length()` and, later, `ArrayList`'s `size()`. Yes, it is inconsistent. Yes, it trips everyone.)

Two ways to walk an array:

```
for (int i = 0; i < a.length; i++) {    // when you need the index i
    System.out.println(i + ": " + a[i]);
}
for (int value : a) {                  // when you just need each value
    System.out.println(value);
}
```

The enhanced `for` gives you *values*, not indices, and you cannot assign back into the array through the loop variable: `value = 99`; changes the copy, not the slot. If you need to modify elements in place, use the counted loop.

The reference idea is the whole ballgame for the rest of the notes. An array variable holds a reference to the array object, which lives on the heap. So:

```
int[] x = {1, 2, 3};
int[] y = x;           // y and x now point at the SAME array
y[0] = 99;
// x[0] is now 99 too -- there is only one array
```

If you actually want a separate copy, ask for one: `int[] z = Arrays.copyOf(x, x.length)`; Keep this picture; 2-D arrays are built entirely out of it.

2. Two-dimensional arrays: really “arrays of arrays”

Some languages give you a true rectangular block of memory. Java does not. A Java 2-D array is a **one-dimensional array whose elements are themselves array references**. Internalize that sentence and everything else falls out of it.

```
int[][] grid = new int[3][4];    // 3 rows, each row a separate int[4]
```

What actually exists: one array of length 3, holding three references, each pointing at its own `int[4]`. That is why:

```
grid.length      // 3    -- number of rows (length of the outer array)
grid[0].length   // 4    -- number of columns in row 0 (length of that inner array)
```

`grid.length` is *rows*, `grid[i].length` is *the width of row i*. Say it out loud a few times; on 2-D exams this is where points quietly leak away.

Indexing is `grid[row][col]`. I always read the first bracket as “which row” and the second as “which column,” but nothing in Java forces that interpretation; you could treat the first index as *x* and the second as *y*. Pick a convention for your program and never waver, because the compiler will not remind you which is which.

Iterating: the standard nested loop, outer over rows, inner over columns.

```

for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[r].length; c++) {    // note grid[r].length, not grid[0].
        length
        grid[r][c] = r * 10 + c;
    }
}

```

I write `grid[r].length` and not `grid[0].length` on purpose: it costs nothing when the grid is rectangular and it saves you when it is not (see ragged arrays below).

Enhanced form, since each element of the outer array *is* a row:

```

for (int[] row : grid) {
    for (int value : row) {
        System.out.print(value + " ");
    }
    System.out.println();
}

```

Printing. `System.out.println(grid)` prints something like `[[I@1b6d3586:` a type tag and a hash, useless. `Arrays.toString(grid)` is only slightly better because it stringifies each *row reference*. The one you want is `Arrays.deepToString(grid)`, which recurses:

```

System.out.println(Arrays.deepToString(grid));    // [[0, 1, 2, 3], [10, 11, 12, 13],
...

```

Initializer syntax works in nested form:

```

int[][] magic = {
    {2, 7, 6},
    {9, 5, 1},
    {4, 3, 8}
};

```

3. Ragged (jagged) arrays

Because each row is an independent array, the rows do not have to be the same length. This is legal and sometimes exactly what you want:

```

int[][] triangle = new int[4][];    // 4 row slots, but NO rows allocated yet
for (int r = 0; r < triangle.length; r++) {
    triangle[r] = new int[r + 1];    // row 0 has 1 slot, row 1 has 2, ...
}

```

Right after `new int[4][]`, every row is `null`: you allocated the outer array of references but none of the inner arrays. This is where `grid[r].length` in the inner loop earns its keep; `grid[0].length` would give you the wrong width for every row but the first (and here would just be wrong, period).

Pascal's triangle, a lower-triangular matrix, or “the games each player played this week” (different counts per player) are all natural ragged arrays.

4. Three-dimensional arrays

Same idea, one more layer. `int[][][]` is an array of 2-D arrays.

```
int[][][] cube = new int[2][3][4];    // 2 layers, each 3x4
cube[1][2][3] = 42;
```

Read the indices outside-in: layer, then row, then column. `cube.length` is the number of layers, `cube[0].length` the rows in layer 0, `cube[0][0].length` the columns. Three nested loops to traverse; `Arrays.deepToString` still prints it.

Honest take: you will write 2-D arrays constantly and 3-D arrays rarely. When 3-D shows up it is usually genuinely three-dimensional data: a stack of image frames (frame, row, column), a voxel world (x, y, z), or a grid of values sampled over time (t, row, column). If you find yourself reaching for four or more dimensions, that is usually a sign the data wants a class or a different structure, not more brackets.

5. The Arrays utility class: stop writing this by hand

`java.util.Arrays` is a box of static helpers. Learn these six and you will rarely hand-roll array bookkeeping again:

- `Arrays.toString(a)` and `Arrays.deepToString(a)`: readable printing (flat and nested).
- `Arrays.sort(a)`: sorts in place, ascending. There is a range version `sort(a, from, to)`.
- `Arrays.fill(a, value)`: sets every slot to `value`. Handy for initializing to something other than the default.
- `Arrays.copyOf(a, newLength)`: returns a new array of the given length (truncated or zero-padded). This is how you “resize” an array: you do not, you make a bigger one and copy. (Remember that when we get to `ArrayList`; it is doing exactly this behind your back.)
- `Arrays.equals(a, b)`: element-by-element comparison. Use this, never `a == b`, to ask whether two arrays hold the same contents (`==` only asks if they are the same object). For nested arrays, `Arrays.deepEquals`.
- `Arrays.binarySearch(a, key)`: fast lookup, but **only valid on a sorted array**. On an unsorted array the result is meaningless, not an error, which makes it a nasty bug.

`Arrays.asList(...)` also lives here; I mention it in the traps section because it does not do quite what its name suggests.

6. ArrayList: a resizable, object-friendly array

An array's length is frozen at creation. Constantly you do not *know* the count ahead of time: you are reading lines from a file, collecting user input, tracking enemies that spawn and die. `ArrayList<E>` is the answer: it wraps an array, and when that backing array fills up, it quietly allocates a bigger

one and copies everything over (exactly the `copyOf` move from above). You get the convenience of “just keep adding” at the cost of a little overhead.

E is the element type, a **type parameter** (generics). You state what it holds and the compiler enforces it:

```
ArrayList<String> names = new ArrayList<>();    // holds Strings; <> infers the type
names.add("Ada");
names.add("Grace");
String first = names.get(0);    // no cast needed -- the compiler knows it's a String
```

Core operations (note the names; they differ from arrays on purpose):

```
list.add(x);                // append to the end
list.add(i, x);             // insert at index i, shifting the rest right
list.get(i);                // read element i (NOT list[i] -- no bracket syntax on an
    ArrayList)
list.set(i, x);             // overwrite element i, returns the old value
list.remove(i);             // remove element at index i, shifting the rest left
list.size();               // how many elements (a method; arrays use the .length field)
list.contains(x);          // membership test
list.indexOf(x);           // first index of x, or -1
list.isEmpty();            // size() == 0
```

Three big syntax differences from arrays, all of which people mix up for a week and then never again: `get` and `set` instead of `[]`, `size()` (method, parentheses) instead of `length` (field, none), and you must declare an element type in angle brackets.

Iterating. Any of these; pick by what you need:

```
for (int i = 0; i < list.size(); i++) { ... list.get(i) ... }    // when you need the
    index
for (String n : list) { ... }                                    // when you just need
    each
list.forEach(n -> System.out.println(n));                        // functional style
```

Autoboxing: how a list of numbers works. A type parameter has to be a class type; you cannot write `ArrayList<int>`. You write `ArrayList<Integer>`, and Java *autoboxes*: primitives are automatically wrapped going in and unwrapped coming out.

```
ArrayList<Integer> nums = new ArrayList<>();
nums.add(7);                // int 7 auto-wrapped to Integer.valueOf(7)
int x = nums.get(0);        // Integer auto-unwrapped back to int
```

Convenient, but not free: every number becomes a heap object. For a few dozen values, who cares. For millions of numbers in a tight loop, that overhead is real and a plain `int[]` is the right call.

Size versus capacity: worth understanding, easy to overthink. *Size* is how many elements you have actually added (`size()`). *Capacity* is how long the backing array currently is, the room you have before the next grow. You rarely touch capacity directly, but knowing it exists explains

the growth behavior: when you add past capacity, the list allocates a larger backing array and copies. In the current OpenJDK implementation the new capacity is about **1.5× the old** (grow by half), *not* double; a lot of textbooks and older notes say “doubles,” and that is the classic detail to double-check against a real JDK rather than take my word for. The default starting capacity is 10, allocated lazily on the first `add`. If you already know you will add 10,000 items, `new ArrayList<>(10000)` skips a pile of intermediate copies.

Converting to and from arrays, because you will need both:

```
Integer[] arr = nums.toArray(new Integer[0]);           // ArrayList -> array
List<String> view = Arrays.asList("a", "b", "c");         // array-ish -> List (fixed
    size!)
List<String> real = new ArrayList<>(Arrays.asList("a","b")); // a genuinely resizable
    copy
```

7. The traps everyone hits once

`remove(int)` versus `remove(Object)` is the sharpest edge in the whole topic. `List` has both `remove(int index)` and `remove(Object o)`. With an `ArrayList<Integer>`:

```
list.remove(2);                // calls remove(int) -- removes the element AT
    INDEX 2
list.remove(Integer.valueOf(2)); // calls remove(Object) -- removes the VALUE 2
```

If you meant “delete the number 2” and wrote `list.remove(2)`, you deleted whatever was at index 2 instead, silently. This one is worth memorizing cold.

Modifying a list while a for-each walks it throws `ConcurrentModificationException`:

```
for (String s : list) {
    if (s.isEmpty()) list.remove(s); // BOOM at runtime
}
```

The fix is an explicit `Iterator` and *its* `remove`, or `list.removeIf(String::isEmpty)`:

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().isEmpty()) it.remove(); // safe
}
```

`Arrays.asList` returns a **fixed-size list** backed by the original array. You can `set` elements but `add` and `remove` throw `UnsupportedOperationException`, and writing through it writes through to the array. When you want a normal, growable list, wrap it: `new ArrayList<>(Arrays.asList(...))`. Same caution for `List.of(...)`, which is fully immutable.

`==` on wrapper objects compares references, not values, and the results look maddeningly inconsistent because Java caches small `Integers` (roughly `-128..127`). Compare values with `.equals` or by unboxing to `int`, never with `==`.

Off-by-one and bounds. `for (i = 0; i <= a.length; i++)` runs one step too far:

`a[a.length]` is out of bounds; the last valid index is `length - 1`. An `ArrayList` throws `IndexOutOfBoundsException` for the same mistake. When a loop blows up on the last iteration, this is the first thing to check.

Fixed size is not the same as immutable

When I call `Arrays.asList(...)` “fixed size,” I mean the *count* is frozen, not that the list is read-only. You can overwrite what is in a slot; you just cannot add slots or remove them.

```
List<String> view = Arrays.asList("a", "b", "c");

view.set(0, "z");    // OK -> [z, b, c]    (changing a slot is fine)
view.add("d");       // UnsupportedOperationException
view.remove(0);      // UnsupportedOperationException
view.clear();        // UnsupportedOperationException
```

The reason is what `Arrays.asList` hands back: **not** a `java.util.ArrayList`, but a thin `List` wrapper sitting directly on top of an array. An array’s length can never change, so any operation that would change the *count* (`add`, `remove`, `clear`) has nowhere to go and throws. Operations that keep the count the same (`set`, `get`, iterating) work normally.

Three tiers worth keeping straight; this one sits in the middle:

- **Fully mutable** (`new ArrayList<>()`): change contents and grow or shrink freely.
- **Fixed size** (`Arrays.asList(...)`): change contents, but the count is frozen.
- **Immutable** (`List.of(...)`): cannot change anything, not even `set`.

One wrinkle that makes “backed by an array” literal: if you pass an actual array *variable* (not loose values), the list is a live view of it. Writing through the list writes into the array, and the reverse:

```
String[] data = {"a", "b", "c"};
List<String> view = Arrays.asList(data);
view.set(0, "z");
System.out.println(data[0]);    // z -- the array changed too
```

That is why, when you want a normal growable list, you wrap it: `new ArrayList<>(Arrays.asList(...))`. Wrapping copies the elements into a genuine resizable list, which breaks the tie to the array and unlocks `add` and `remove`.

8. Array or ArrayList? The decision you will actually make

Reach for a **plain array** when the size is fixed and known, when you are storing primitives in a performance-sensitive spot, when the data is naturally multi-dimensional (grids, boards, matrices; there is no clean “2-D `ArrayList`”, you would nest, and it gets ugly fast), or when you are interoperating with an API that hands you arrays.

Reach for an `ArrayList` when the count grows or shrinks at runtime, when you do not know the count up front, when you want the convenience methods (`contains`, `indexOf`, `remove`, `removeIf`) and do not want to manage a backing array by hand. This is the default choice for most everyday “a bunch of things” needs.

Rule of thumb I actually use: *fixed grid of primitives* → *array*; *growing pile of objects* → *ArrayList*. When neither obviously wins, start with the **ArrayList** for the readability and switch to an array only if a profiler tells you to.

9. Where this lives in the book

In *Core Java, Volume I* (13th ed.), arrays are covered near the end of the fundamentals material (the chapter on basic language constructs), and **ArrayList**, “generic array lists,” is introduced with generics and inheritance, with the fuller **List** and **Collections** treatment later in the collections chapter. *Volume II* goes deeper on the **Collections** framework if you want it. I am deliberately not pinning exact section numbers here because they drift between printings; flip to the arrays and collections entries in the index and you will land in the right place. (And if a number I *do* say in class turns out wrong, that is exactly the kind of thing to flag for me.)

Last thing: read these once before class and once after. The first pass makes the live examples make sense; the second pass is where it sticks. And I mean it about reporting errors: a note that is been wrong for three semesters is only wrong because nobody told me.